

Chatbot Security Measures

1. Introduction

This Python script extends the previously implemented Chatbot Conversation Script by focusing on *security measures*. While the conversation script provided branching logic for wellness interactions, this script ensures the chatbot is safe, robust, and resistant to misuse.

Key requirements:

- Input sanitisation logic (profanity and unsafe content removal).
- Explanation and demonstration of rate-limiting and response delays.

2. Input Sanitization

Objectives

- Prevent the chatbot from displaying offensive or harmful text.
- Neutralize unsafe inputs (e.g., scripts, repeated spam, or self-harm mentions).
- Ensure only safe content is processed and shown back to users.

Logic Implemented

- **Profanity masking:** Replace middle letters of flagged words with *.
Example: "shit" → "s*t".
- **HTML/script escaping:** Convert <script>...</script> to safe characters.
Prevents XSS or injection attacks in UI contexts.
- **Spam collapse:** Detect repeated characters (soooooo → sooo).
- **Unsafe keyword detection:** Identify critical categories:
 - Crisis/self-harm → escalation message triggered.
 - Violence → escalation message triggered.

Evidence

- **Screenshot:** security_sanitize_unsafe.png

```
--- Input Sanitization & Unsafe Detection ---
Raw: Hello, how are you?
Sanitized: Hello, how are you?
>> Safe for chatbot response.

-----
Raw: You are a damn fool!
Sanitized: You are a d**n fool!
>> Safe for chatbot response.

-----
Raw: I will kill myself tonight...
Sanitized: I will kill myself tonight...
>> Escalation: Crisis support needed.

-----
Raw: This is soooooooooo goood!!!!
Sanitized: This is sooo goood!!!
>> Safe for chatbot response.

-----
Raw: <script>alert('hack');</script>
Sanitized: &lt;script&gt;alert(&#x27;hack&#x27;);&lt;/script&gt;
>> Safe for chatbot response.

-----
Raw: I might attack someone
Sanitized: I might attack someone
>> Escalation: Violence-related content detected.

-----
Raw: Regular safe message
Sanitized: Regular safe message
>> Safe for chatbot response.

-----
```

○ Screenshot: security_demo_full.png & security_code_snippet.png

```
Welcome chatbot_security.py security_demo_output.txt x
security_demo_output.txt
2 Chatbot Security Measures Demo
3 =====
4
5 --- Input Sanitization & Unsafe Detection ---
6 Raw: Hello, how are you?
7 Sanitized: Hello, how are you?
8 >> Safe for chatbot response.
9 =====
10 Raw: You are a damn fool!
11 Sanitized: You are a d**n fool!
12 >> Safe for chatbot response.
13 =====
14 Raw: I will kill myself tonight...
15 Sanitized: I will kill myself tonight...
16 >> Escalation: Crisis support needed.
17 =====
18 Raw: This is soooooooooo goood!!!!
19 Sanitized: This is sooo goood!!!
20 >> Safe for chatbot response.
21 =====
22 Raw: <script>alert('hack');</script>
23 Sanitized: &lt;script&gt;alert(&#x27;hack&#x27;);&lt;/script&gt;
24 >> Safe for chatbot response.
25 =====
26 Raw: I might attack someone
27 Sanitized: I might attack someone
28 >> Escalation: Violence-related content detected.
29 =====
30 Raw: Regular safe message
31 Sanitized: Regular safe message
32 >> Safe for chatbot response.
33 =====
34
35 --- Rate Limiting Demo ---
36 Request 1: Allowed
37 Request 2: Allowed
38 Request 3: Allowed
39 Request 4: BLOCKED (too many requests)
40 Request 5: BLOCKED (too many requests)
41
42 --- Response Delay Demo ---
43 Bot is thinking... Done! (after small delay)
```

```
def sanitize_input(user_text: str) -> str:
    """
    Sanitize raw user input:
    - Escape HTML/script injections
    - Collapse whitespace
    - Mask profanities
    - Reduce spammy repetitions
    """
    if not user_text:
        return ""
    # Step 1: Normalize and escape HTML
    safe = html.escape(user_text.strip())
    # Step 2: Collapse extra whitespace
    safe = re.sub(r"\s+", " ", safe)
    # Step 3: Mask profanities
    safe = mask_profanity(safe)
    # Step 4: Reduce spam (e.g., "aaaaaa" -> "aaa")
    for pat in SPAM_PATTERNS:
        safe = re.sub(pat, r"\1\1\1", safe)
    return safe

def is_unsafe(user_text: str) -> Tuple[bool, str]:
    """
    Check for unsafe content.
    Returns (True/False, reason).
    Reasons: "crisis", "violence"
    """
    low = (user_text or "").lower()
    if any(k in low for k in CRISIS_KEYWORDS):
        return True, "crisis"
    if any(k in low for k in VIOLENCE):
        return True, "violence"
    return False, ""
```

3. Unsafe Content Escalation

Objectives

- Recognise when users are at risk (e.g., expressing suicidal thoughts).
- Provide appropriate escalation instead of continuing the normal chat flow.

Logic Implemented

- Keyword detection for phrases like “kill myself”, “hurt myself”, “end my life”.
- Escalation response returns a supportive message with helpline details (AU context: Lifeline, Beyond Blue).
- Violence-related words trigger a warning about unsafe input.

Evidence

- Crisis escalation example: “I will kill myself tonight...”.
- Violence escalation example: “I might attack someone”.

4. Rate Limiting

Objectives

- Prevent abuse by blocking excessive or automated requests.
- Ensure fairness and responsiveness for genuine users.

Logic Implemented

- The sliding-window limiter allows up to 3 requests per 10 seconds.
- Further requests within that window are blocked with a warning.
- In production, this would extend to persistent storage (e.g., Redis) for multi-user tracking.

Evidence

- **Output shows:**
 - Request 1: Allowed
 - Request 2: Allowed
 - Request 3: Allowed
 - Request 4: BLOCKED
 - Request 5: BLOCKED
- **Screenshot:** security_rate_limit.png.

```
--- Rate Limiting Demo ---
Request 1: Allowed
Request 2: Allowed
Request 3: Allowed
Request 4: BLOCKED (too many requests)
Request 5: BLOCKED (too many requests)
```

5. Response Delay

Objectives

- Add small, natural pauses to discourage bot spamming.
- Improve the realism of chatbot responses.

Logic Implemented

- Randomised delay between 200–700 ms before sending a reply.
- Makes scripting abuse less effective while maintaining usability.

Evidence

- **Output:** Bot is thinking... Done! (after small delay)
- **Screenshot:** security_delay.png.

```
--- Response Delay Demo ---
Bot is thinking... Done! (after small delay)
```

6. Evidence: Complete Run

The program was executed using the following command:

- `python3 chatbot_security.py | tee security_demo_output.txt`
- **Screenshot:** Run_Commands.png

```
bhanupratap@Bhanus-MacBook-Pro Chatbot Security % python3 chatbot_security.py | tee security_demo_output.txt
=====
Chatbot Security Measures Demo
=====

--- Input Sanitization & Unsafe Detection ---
Raw: Hello, how are you?
Sanitized: Hello, how are you?
>> Safe for chatbot response.

-----
Raw: You are a damn fool!
Sanitized: You are a d**n fool!
>> Safe for chatbot response.

-----
Raw: I will kill myself tonight...
Sanitized: I will kill myself tonight...
>> Escalation: Crisis support needed.

-----
Raw: This is soooooooo goood!!!!
Sanitized: This is sooo goood!!!
>> Safe for chatbot response.

-----
Raw: <script>alert('hack');</script>
Sanitized: &lt;script&gt;alert(&#x27;hack&#x27;);&lt;/script&gt;
>> Safe for chatbot response.

-----
Raw: I might attack someone
Sanitized: I might attack someone
>> Escalation: Violence-related content detected.

-----
Raw: Regular safe message
Sanitized: Regular safe message
```

This created a reproducible text transcript (security_demo_output.txt) containing all outputs.

7. Discussion

- **Strengths:**
 - Covers a wide variety of unsafe inputs (profanity, spam, scripts, crisis/violence).
 - Demonstrates escalation paths with real-world helpline references.
 - Implements simple but effective rate limiting and delay mechanisms.
 - Fully reproducible, standalone script.
- **Limitations:**
 - Profanity list is small; production systems require curated or third-party moderation APIs.
 - Unsafe keyword detection is rule-based and may miss subtle cases.
 - In-memory rate limiter resets per run; real deployment needs persistent storage.
- **Future Improvements:**
 - Use ML/NLP models for nuanced content moderation.
 - Add logging/auditing for flagged events.
 - Localise escalation numbers/resources for global use.

8. Conclusion

This script successfully demonstrates input sanitisation, unsafe content escalation, rate limiting, and response delays. Together, these measures strengthen chatbot resilience and user safety, ensuring the system cannot be easily misused or exploited.